

R05 - The Practical Test Pyramid

- <https://martinfowler.com/articles/practical-test-pyramid.html>

Importância de automação de testes

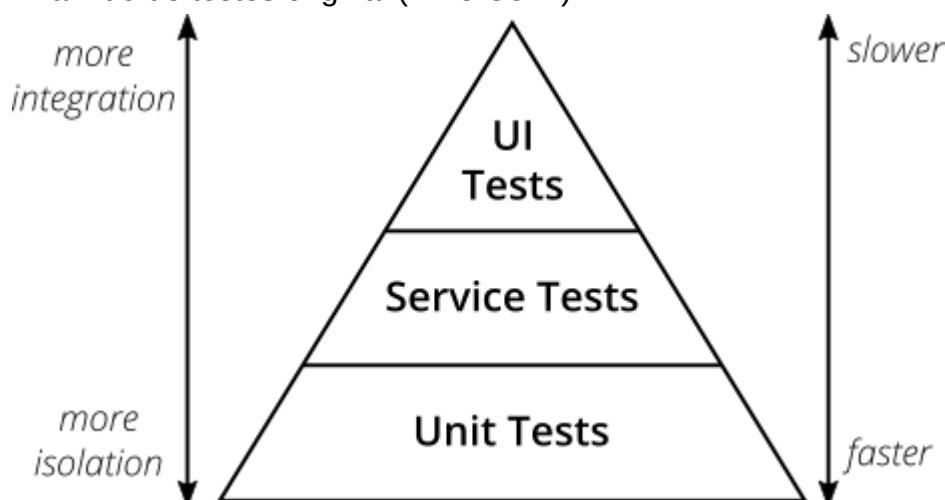
- Aumentar a velocidade sem afetar sua qualidade
- Entrega contínua
 - O software pode ser lançado em produção a qualquer momento
 - Pipeline de compilação para testar automaticamente e implantá-lo nos ambientes de teste e produção

Ferramentas e bibliotecas

- [JUnit](#): executor de testes
- [Mockito](#): Simular dependências
- [Wiremock](#): Substituir serviços externos por stubs
- [Pact](#): Escrever testes de CDC
- [Selenium](#): escrever testes de ponta a ponta orientados pela interface do usuário
- [REST-assured](#): Escrever testes ponta a ponta orientados por API REST

Pirâmide de testes

- Metáfora para agrupar testes de software em blocos de diferente granularidade:
 - Pirâmide de testes original (Mike Cohn):



- Use a original como base para criar sua própria pirâmide de testes mais detalhada
 - 2 ideias principais: escreva testes com granularidades diferentes, quanto mais alto o nível, menos testes
 - Evite o cone de sorvete: muitos testes lentos de alto nível, pouca unidade — difícil de manter
- **Testes de Unidade**

- Possuem o menor escopo de todos os conjuntos de testes
- Testes de múltiplos parâmetros em uma única função (linguagens funcionais).
- **Testes unitários isolados:** Simulam todos os colaboradores
- **Testes unitários interativos:** Permitem interação com colaboradores reais
- Test Doubles: Termo genérico para trocar um objeto em produção por um falso que devolve valores similares aos reais para auxiliar na escrita de testes.
 - Podem ser usados para simular partes inteiras de um sistema (Não só em testes de unidade)
 - **Mocks:** Usam expectativas pré-programadas sobre as chamadas que devem receber. Podem levantar erros diante de chamadas inesperadas.
 - **Stubs:** Respostas pré-programadas às chamadas feitas durante o teste
 - +3: Dummy, Fake, Spies

• O que testar?

- Para qualquer classe, use a regra **Uma classe de teste por classe em produção**
- Uma classe de teste de unidade deve testar pelo menos **a interface pública de uma classe**. Métodos privados são detalhe de implementação, teste pela interface pública.
- Devem **garantir que todos os caminhos de código não triviais sejam testados**, porém **não devem estar intimamente ligados à implementação**, que podem quebrar durante refatorações
- Teste comportamentos observáveis ("Se eu inserir os valores `x` e `y`, o resultado será `z` ?")
- Não testes códigos triviais, não tem nenhum benefício.

• Estrutura de teste

1. Prepare os dados de teste (Arrange / Given)
2. Chame o método que está sendo testado (Act / When)
3. Verifique se os resultados esperados estão sendo retornados (Assert / Then)
 - Testes com essa estrutura tendem a ser menores e mais expressivas

• Testes de Integração / Serviços

- Testam a integração com **um ponto por vez fora da sua aplicação**. Mais lentos e mais difícil de serem escritos. Podem ser whitebox.
- Execute os testes com a própria ferramenta externa (Ex: banco de dados). Prefira dependências locais, se não for possível, prefira executar uma instância dedicada para testes.
- Substitua outras ferramentas que não estão sendo testadas por Test Doubles para evitar lentidão
- Escreva testes de integração para todos **pedaços de código em que você serializa ou desserializa dados** (Transformar dados em formato adequado para armazenamento ou transmissão)

- API REST
- Leitura e escrita em banco de dados
- Outras APIs
- Leitura e escrita de filas
- Escrita ao sistema

• Testes de Contratos

- **Contrato:** Especificação da API para usuários
- **Testes de Contrato Orientados aos Usuários (testes CDC)**
 - Os consumidores conduzem o desenvolvimento dos testes do contrato, os provedores desenvolvem a API se baseando em todos os testes CDC.
 - Implementa apenas o que é realmente necessário
 - A interface entre times funciona o tempo todo
 - Fluxo: consumidor escreve expectativas e publica, provedor executa continuamente e mantém verde, conversam se quebrar
- **Testes de Consumidor**
 - O consumidor define expectativas para o contrato, o provedor desenvolve.
- **Testes de Provedores**
 - O provedor lê as expectativas publicadas e verifica se sua API as atende

• Testes de UI

- Testam se a interface da sua aplicação está funcionando corretamente.
- Não precisa ser teste completo do front + back, mocke os dados. Teste de UI não precisa ser de ponta a ponta
- Ferramentas para teste de layout: Usam Selenium para tirar foto e comparar com anteriores
 - [Galen](#)
 - [Lineup](#) e [JLineup](#)
- Fatores de usabilidade e de "parece bom" não são possíveis automatizar. Use [Testes exploratórios](#), [Testes de usabilidade](#) e demonstrações para usuários

• Testes de Ponta a Ponta

- Testam a aplicação através da interface do usuário
- Geralmente são instáveis, ou falham por motivos inesperados e imprevisíveis. Provavelmente falso positivo.
- Necessitam de bastante manutenção e são muito lentos. Reduza a quantidade de Testes Ponta a Ponta ao mínimo possível, foque nas jornadas de maior valor para o usuário
- [Selenium](#) e [Protocolo Webdriver](#) permitem automatizar os testes através de um navegador headless, executando cliques, acessando dados e verificando o estado da interface

• Testes de Aceitação

- Testar se um software funciona corretamente da perspectiva do usuário
- Pode ser em diferentes níveis de granularidade, não precisa ser só no topo da pirâmide
- Foco em comportamento: dado um contexto, quando o usuário age, então o resultado esperado acontece

• **Testes Exploratórios**

- Abordagem de Testes Manuais que enfatiza a liberdade e criatividade do testador a encontrar problemas de qualidade e execução de sistema.
- Tente quebrar sua aplicação. Tente provocar problemas e erros na sua aplicação. Documente tudo o que encontrar para depois. Verifique por bugs, problemas de design, tempo de resposta lentos, mensagens errôneas ou faltando. Qualquer coisa que te irrita como usuário

Pipeline, duplicação e código limpo

- Terminologia varia entre times e empresas, combine os nomes e o escopo de cada tipo de teste no seu time
- Coloque testes rápidos no início do pipeline para feedback rápido, testes lentos e amplos depois
- Evite duplicação: empurre os testes para o nível mais baixo possível, se um teste alto falha sem baixo falhar, crie o teste baixo
- Código de teste importa tanto quanto produção: 1 condição por teste, use Arrange/Act/Assert, priorize legibilidade